Reusable Django Apps and Python Packaging

Python Django Backend Developer Notes

---

1. What is a Reusable Django App?

A reusable Django app is an independent Django application that can be used in multiple Django projects with minimal changes.

Examples

- Authentication and authorization
- Common API responses
- Pagination utilities
- Custom permissions
- Logging and notifications
- Audit trail

Benefits

- Code reusability
- Less code duplication
- Easy maintenance
- Better testing
- Faster development

Example: Create one common API response app and use it in E-commerce, Blog, and School Management projects.

---

2. Create a Django Project

Step 1: Create a virtual environment

```
mkdir django-reusable-learning
cd django-reusable-learning

python -m venv venv
```

Activate on Windows:

```
venv\Scripts\activate
```

Step 2: Install dependencies

```
python -m pip install django djangorestframework
```

Step 3: Create the project and app

```
django-admin startproject config .
python manage.py startapp common_api
```

Project structure

```
django-reusable-learning/
├── manage.py
├── config/
│   ├── settings.py
│   └── urls.py
└── common_api/
    ├── __init__.py
    ├── apps.py
    ├── views.py
    ├── tests.py
    └── responses.py
```

---

3. Register the Django App

In `config/settings.py`:

```
INSTALLED_APPS = [
    # Default Django apps...
    "rest_framework",
    "common_api",
]
```

---

4. Create Reusable API Response Functions

Create `common_api/responses.py`:

```
from rest_framework.response import Response


def success_response(
    data=None,
    message="Success",
    status_code=200,
):
    return Response(
```

```
        {
            "success": True,
            "message": message,
            "data": data,
        },
        status=status_code,
    )


def error_response(
    message="Something went wrong",
    status_code=400,
    data=None,
):
    return Response(
        {
            "success": False,
            "message": message,
            "data": data,
        },
        status=status_code,
    )
```

Usage

```
from common_api.responses import success_response

return success_response(
    data={"name": "Faruk"},
    message="User retrieved successfully",
)
```

Output

```
{
    "success": true,
    "message": "User retrieved successfully",
    "data": {
        "name": "Faruk"
    }
}
```

Key point: Write the function once and reuse it in multiple API views and projects.

---

5. Use the Functions in API Views

In `common_api/views.py`:

```
from rest_framework.views import APIView

from .responses import success_response, error_response


class DemoAPIView(APIView):

    def get(self, request):
        user = {
            "id": 1,
            "name": "Faruk",
            "role": "Backend Developer",
        }

        return success_response(
            data=user,
            message="User retrieved successfully",
        )


class ErrorDemoAPIView(APIView):

    def get(self, request):
        return error_response(
            message="User not found",
            status_code=404,
        )
```

In `config/urls.py`:

```
from django.contrib import admin
from django.urls import path

from common_api.views import DemoAPIView, ErrorDemoAPIView

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/demo/", DemoAPIView.as_view()),
    path("api/error/", ErrorDemoAPIView.as_view()),
]
```

Run the development server:

```
python manage.py runserver
```

Test these URLs:

- http://127.0.0.1:8000/api/demo/
- http://127.0.0.1:8000/api/error/

---

6. Test the Django App

In `common_api/tests.py`:

```
from rest_framework.test import APITestCase


class DemoAPITest(APITestCase):

    def test_demo_api(self):
        response = self.client.get("/api/demo/")

        self.assertEqual(response.status_code, 200)
        self.assertTrue(response.data["success"])
        self.assertEqual(
            response.data["data"]["name"],
            "Faruk",
        )

    def test_error_api(self):
        response = self.client.get("/api/error/")

        self.assertEqual(response.status_code, 404)
        self.assertFalse(response.data["success"])
```

Run tests:

```
python manage.py test common_api
```

Purpose of testing: Verify expected behavior and prevent regressions when the code changes.

---

7. What is Python Packaging?

Python packaging is the process of organizing, building, and distributing Python code so that other projects can install and use it.

Important terms

- **Package:** A collection of importable Python modules.
- **Module:** A Python file, such as `responses.py`.
- **pip:** A Python package installer.
- **PyPI:** Python Package Index, where Python packages can be published.
- **pyproject.toml:** Project metadata and build configuration.
- **Wheel:** A built distribution format with the `.whl` extension.
- **Source distribution:** An archive containing source files and package metadata.

Difference

- Reusable Django app = reusable application code.
- Python packaging = preparing and distributing that code as an installable package.

---

8. Create a Python Package

Recommended structure:

```
django-common-toolkit/
├── pyproject.toml
├── README.md
├── LICENSE
├── src/
│   └── common_api/
│       ├── __init__.py
│       ├── apps.py
│       └── responses.py
└── tests/
    └── test_responses.py
```

The distribution name and import name can be different:

- Distribution name: `django-common-toolkit-faruk`
- Import name: `common_api`

Copy the reusable app code into the `src/common_api/` directory.

---

9. Configure pyproject.toml

Create `pyproject.toml` in the package root:

```
[build-system]
requires = ["setuptools>=68"]
```

```
build-backend = "setuptools.build_meta"

[project]
name = "django-common-toolkit-faruk"
version = "0.1.0"
description = "Reusable API utilities for Django"
readme = "README.md"
requires-python = ">=3.10"
dependencies = [
    "Django>=4.2",
    "djangorestframework>=3.14",
]

[tool.setuptools.packages.find]
where = ["src"]
```

Important fields

- `name` : Distribution package name.
- `version` : Current release version.
- `description` : Short package description.
- `readme` : Project documentation.
- `requires-python` : Minimum supported Python version.
- `dependencies` : Required external libraries.
- `build-system` : Build tools and backend.
- `where` : Location of Python packages.

Note: Check package name availability and test supported dependency versions before publishing.

---

10. Install the Package Locally

From the package root:

```
python -m pip install -e .
```

`-e` means editable installation.

Editable installation is useful during development because source changes are available without repeatedly reinstalling the project.

In another Django project, add the app to `INSTALLED_APPS` :

```
INSTALLED_APPS = [
    # Other apps...
    "common_api",
]
```

Import its function:

```
from common_api.responses import success_response
```

The package must be installed in that project's active Python environment.

---

11. Build the Package

Install the build tool:

```
python -m pip install build
```

Build distributions:

```
python -m build
```

The generated files will usually be placed in `dist/`:

```
dist/
├── django_common_toolkit_faruk-0.1.0-py3-none-any.whl
└── django_common_toolkit_faruk-0.1.0.tar.gz
```

Distribution formats

- `.whl` = Wheel distribution.
- `.tar.gz` = Source distribution.

These files can be used to install or distribute the package.

---

12. Validate the Package

Install Twine:

```
python -m pip install twine
```

Validate distribution metadata:

```
python -m twine check dist/*
```

Also run your automated tests before publishing.

---

13. Publish to PyPI

Steps:

1. Create a PyPI account.
2. Prepare README, license, and package metadata.
3. Run automated tests.
4. Build the package.
5. Validate the distributions.
6. Test publishing and installation with TestPyPI.
7. Publish to PyPI using a supported authentication method.
8. Install the released package in a separate Django project.

Official resources:

- Python Packaging Guide: https://packaging.python.org/en/latest/tutorials/packaging-projects/
- TestPyPI: https://test.pypi.org/
- PyPI: https://pypi.org/

Security: Never commit API tokens or passwords to source control.

---

14. Semantic Versioning

Format: `MAJOR.MINOR.PATCH`

Version change	Meaning	Example
MAJOR	Breaking changes	1.0.0 → 2.0.0
MINOR	Backward-compatible features	1.0.0 → 1.1.0
PATCH	Backward-compatible fixes	1.0.0 → 1.0.1

---

15. Best Practices

- Follow the DRY principle.
- Keep each app focused on a specific responsibility.
- Avoid project-specific hardcoded settings.
- Write clear documentation.
- Add automated tests.
- Declare dependencies accurately.
- Use version control and release tags.
- Test compatibility before releasing.
- Never expose API tokens or secrets.
- Check migrations when the reusable app contains database models.

---

16. Quick Revision Questions

1. What is a reusable Django app?
2. What is Python packaging?
3. What is the difference between a Django app and a Python package?
4. What is `pyproject.toml`?
5. What is the purpose of `pip`?
6. What is PyPI?
7. What is a wheel file?
8. What does `pip install -e .` do?
9. Why are automated tests important?
10. What is Semantic Versioning?

Final Learning Goal

Build a reusable Django app, test its API responses, package it with `pyproject.toml`, create distribution files, and install it in another Django project.

Recommended next project: `django-common-toolkit` with reusable API responses, pagination, exception handling, logging, automated tests, and versioned releases.